

PA3 Report

1. Design

I designed Entity A based on the textbook state diagram and based on my PA2. Entity A initializes next_seqnum, seqnum_limit, window_size is the half of seqnum_limit, base, list, make_seqnum, timeout.

In the output, get the message into the packet with make_seqnum, then checksum, next append the packet into the list, then increment make_seqnum with module by seqnum_limit. Then use a while loop that $\text{next_seqnum} < \text{base} + \text{window_size}$ and check the next_seqnum is in the length of the list. In the while loop, send the current packet to layer 3, then increment the next_seqnum. Next check if the base is equal to next_seqnum, then start the timer.

In the input, use an if loop to check the sum is correct from Entity B, then use a for loop with i that loop from base to base + window_size. Then check i is in the length of list and the current seqnum is equal to packet acknum, next increment the base by 1 and break the for loop. The next step is to check if the base is equal to make_seqnum then stop the timer, else stop the timer first then start the timer. After all these, use the same while loop from output to keep sending packets to layer 3.

In the time interrupt, use the same for loop from input, then use if statement to check i is in the range of the list, and keep sending packets to layer 3. After it sends all the packets, start the timer.

My Entity B for PA3 remains the same from PA2. I designed Entity B based on the textbook state diagram. Entity B receives the packet from Entity A and to check the checksum and seqnum are correct. If they are correct, it sends the message to layer 5, then increments the seqnum. Always send acknum to Entity A.

2. Testing

I added print statements and break points to determine what is going wrong. Based on the print statements, I can get ideas with which packets didn't run correctly. I also run it on the GradeScope for testing. I also run the terminal code to check if A and B send and receive enough messages.

But it can still be hard to debug sometimes. For example, I had my idea formatted into my code, then it runs fine without any loss or corruption. But after I tried to add some loss or corruption separately for testing, it won't receive or send enough messages anymore. From the print statement, I can tell that my input has not received the correct acknum or keep getting the same acknum after a while from Entity B with loss or corruption. Then I went to find TA help, one of the TAs helped me find the error that it was I need a condition check before I start the timer in my while loop. Shout out to all the TA.

Then I used "The Simulated Network Environment" from PA2 to check that during the loss or corruption or both, within n number of messages to simulate, the layer 3 packets sent by A is going to be greater than n because there is loss or corruption or both. Next I check the layer 5 messages delivered by B should be as close to n as possible. To make sure that I receive and send enough messages with the condition of loss or corruption or both. After I felt confidence with my code, I sent it on GradeScope to check. If there are still errors, then I try to go back to debug again and play around with timeout value.

3. Output

```
PS C:\Users\Jiaku\Downloads> python3 rdtsim.py -n 40 -d 100 -z 4 -l  
0.05 -c 0.05 -s 57575757578 -v 2
```

SIMULATION CONFIGURATION

```
-----  
(-n) # layer5 msgs to be provided:    40  
(-d) avg layer5 msg interarrival time: 100.0  
(-z) transport protocol seqnum limit:  4  
(-l) layer3 packet loss prob:         0.05  
(-c) layer3 packet corruption prob:    0.05  
(-s) simulation random seed:          57575757578  
-----
```

==== SIMULATION BEGINS

```
    TO_LAYER3: packet being corrupted  
    TO_LAYER3: packet being lost  
    TO_LAYER3: packet being lost  
    TO_LAYER3: packet being corrupted  
    TO_LAYER3: packet being corrupted  
    TO_LAYER3: packet being corrupted  
    TO_LAYER3: packet being lost  
    TO_LAYER3: packet being corrupted
```

==== SIMULATION ENDS

SIMULATION SUMMARY

```
-----  
# layer5 msgs provided to A:    40  
# elapsed time units:           4378.443552913212  
  
# layer3 packets sent by A:     57  
# layer3 packets sent by B:     54  
# layer3 packets lost:          3  
# layer3 packets corrupted:      5  
# layer5 msgs delivered by A:    0  
# layer5 msgs delivered by B:   39
```

layer5 msgs by B/elapsed time: 0.008907274817794833

In this test, it has the information of my terminal code above. It provided 40 messages and more than 20 messages were successfully transferred from sender to receiver. Also while the trace level is 2, it did print three packets lost and five packets corrupted which is the same as simulation summary. Then the layer 3 packets sent by A and B needs to send more packets than 40. In the end I can tell from layer 5 messages delivered by B is 39, which is very close to 40. That means my protocol correctly recovered from packet loss and corruption.

An example of 1000 messages.

```
PS C:\Users\Jiaku\Downloads> python3 rdtsim.py -n 1000 -d 100 -z 4  
-l 0.1 -c 0.1 -s 57575757578
```

SIMULATION CONFIGURATION

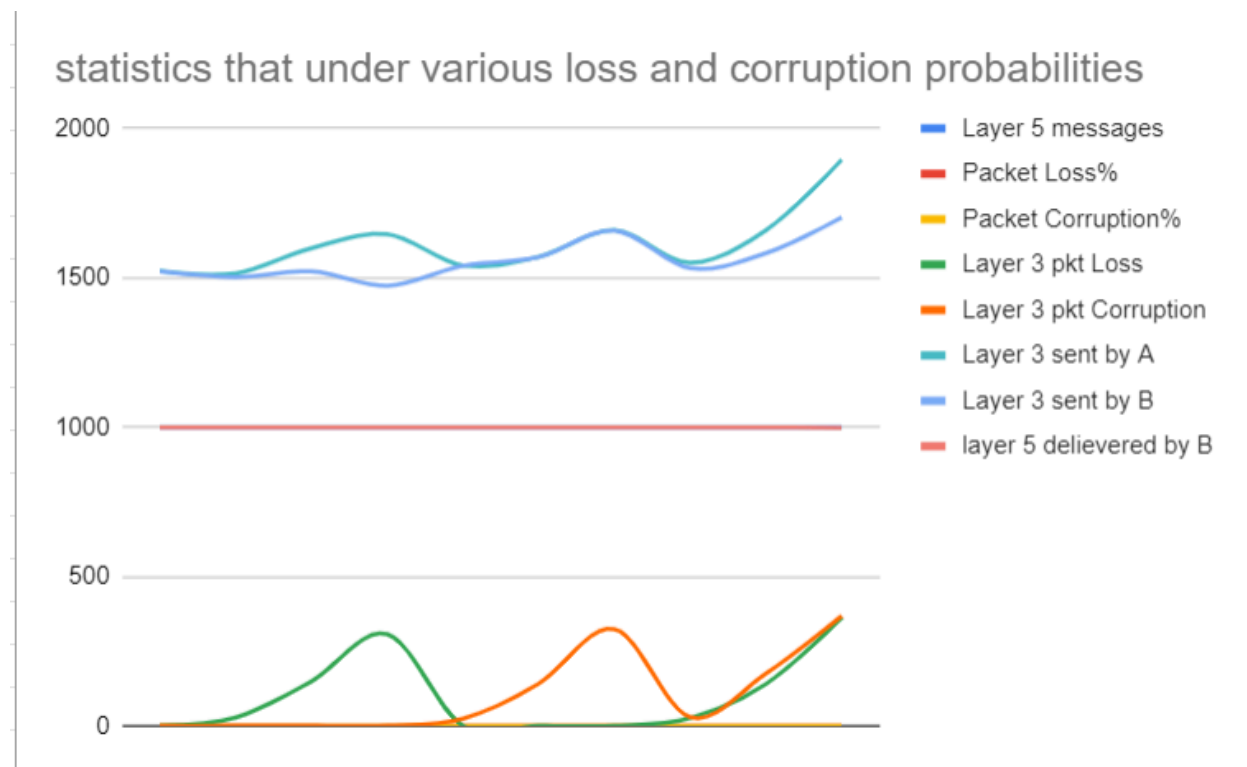
```
-----  
(-n) # layer5 msgs to be provided:    1000  
(-d) avg layer5 msg interarrival time: 100.0  
(-z) transport protocol seqnum limit:  4  
(-l) layer3 packet loss prob:         0.1  
(-c) layer3 packet corruption prob:    0.1  
(-s) simulation random seed:          57575757578  
-----
```

SIMULATION SUMMARY

```
-----  
# layer5 msgs provided to A:    1000  
# elapsed time units:           97108.41652470984  
  
# layer3 packets sent by A:     1895  
# layer3 packets sent by B:     1702  
# layer3 packets lost:          362  
# layer3 packets corrupted:     368  
# layer5 msgs delivered by A:    0  
# layer5 msgs delivered by B:   998  
# layer5 msgs by B/elapsed time: 0.010277173037272755
```

I did many runs with different loss and corruption probability. Here is the sheet below.

	A	B	C	D	E	F	G	H	I
1	Layer 5 message	Packet Loss%	Packet Corruptio	Layer 3 pkt Loss	Layer 3 pkt Corr	Layer 3 sent by	Layer 3 sent by I	layer 5 delievered by B	
2	1000	0	0	0	0	1523	1522	999	
3	1000	0.01	0	28	0	1515	1502	999	
4	1000	0.05	0	149	0	1599	1522	999	
5	1000	0.1	0	307	0	1646	1473	999	
6	1000	0	0.01	0	24	1541	1540	999	
7	1000	0	0.05	0	141	1570	1569	999	
8	1000	0	0.1	0	324	1659	1658	999	
9	1000	0.01	0.01	26	29	1550	1532	999	
10	1000	0.05	0.05	140	175	1659	1582	999	
11	1000	0.1	0.1	362	368	1895	1702	998	
12									



The statistics seem reasonable, the layer 3 messages sent by A and B will raise while either the Layer 3 packet loss or corruption or both raise. The simulator does seem to be functioning correctly, the packet loss or corruption number is the probability with the number that combines Layer 3 messages sent by A and B. The protocol seems working well because the layer 5 delivered by B number looks like on the same line with Layer 5 messages simulated at the sender.